Introduction — Développer en TypeScript avec GitHub Copilot

Bienvenue dans la formation **GitHub Copilot** pour développeurs **TypeScript**.

Cette formation couvre l utilisation de GitHub Copilot au quotidien : completions inline, Chat, personnalisation du dépôt et bonnes pratiques de validation du code généré.

Public et prérequis

PRÉREQUIS	DÉTAIL
Développement	Savoir lire et écrire du code **TypeScript**
Git / GitHub	Compte GitHub, notions de dépôt et commit
Éditeur	Visual Studio Code (recommandé)
Copilot	Abonnement actif (Individual, Business ou Enterprise)

Objectifs globaux

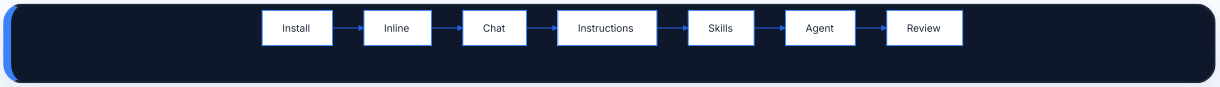
- Utiliser les completions inline avec des prompts précis.
- Maîtriser Copilot Chat (Ask, Edit, Plan, Agent).
- Personnaliser le dépôt avec Instructions et Skills.
- Configurer path-specific, commits et revues assistées.
- Appliquer une checklist de validation sur le code généré.

Organisation — 6 modules

MODULE	THÈME	DURÉE
1	Introduction à GitHub Copilot	2 h
2	Completions inline et contexte	3 h
3	Chat et modes d'interaction	3 h
4	Agent, Skills et Instructions	4 h
5	Path-specific, commit et review	3 h
6	Bonnes pratiques et productivité	2 h

Chaque module comprend un **cours**, des **exercices** et une **correction**.

GRAPHIQUE 1



Prochaine étape : Module 1 — Introduction

Module 1 — Introduction à GitHub Copilot

Ce module pose les bases de GitHub Copilot pour le développement en **TypeScript** : comprendre l'outil, choisir le bon mode, et vérifier l'installation dans VS Code.

Durée indicative : 2 h.

Objectifs

- Définir GitHub Copilot et son rôle de pair-programmer virtuel.
- Distinguer inline, Chat, Agent et CLI.
- Installer et configurer Copilot dans VS Code.
- Consulter les statistiques d'utilisation.

Définition — GitHub Copilot

Assistant IA intégré à l'éditeur. Analyse le **contexte** (fichiers, commentaires, sélection) et propose des **suggestions** en temps réel.

1.1 Qu'est-ce que GitHub Copilot ?

GitHub Copilot est un assistant de programmation basé sur l'intelligence artificielle, développé par GitHub en collaboration avec OpenAI. Il fonctionne comme un **pair-programmer virtuel** intégré directement dans l'éditeur de code.

Principe de fonctionnement :

- Copilot analyse le contexte du code en cours d'écriture (fichiers ouverts, commentaires, noms de variables)
- Il génère des suggestions de code en temps réel, directement dans l'éditeur
- Le modèle sous-jacent a été entraîné sur des milliards de lignes de code provenant de dépôts publics GitHub
- Il est particulièrement efficace en TypeScript grâce à l'écosystème massif open source (React, Node.js, NestJS, Deno, etc.)

Ce que Copilot n'est pas :

- Ce n'est pas le compilateur TypeScript (`tsc`) ni ESLint
- Il ne garantit pas que le code généré est correct ou sécurisé
- Il ne remplace pas la compréhension de TypeScript et du typage statique

1.2 Les différentes versions — du complétion à l'agent

GitHub Copilot propose plusieurs **niveaux d'autonomie**. La formation s'articule autour du mode **Agent** et de sa personnalisation (**Instructions**, **Skills**), tout en conservant les **completions inline** pour l'écriture au fil de l'eau.

VERSION	NIVEAU D'AUTONOMIE	DESCRIPTION	USAGE PRINCIPAL
Copilot (inline)	Faible	Suggestions de code directement dans l'éditeur	Complétion au quotidien, boilerplate
Copilot Chat	Moyen	Conversation (Ask, Edit, Plan)	Questions, explications, refactoring
Mode Agent	Élevé	Planifie, modifie plusieurs fichiers, exécute	Tâches multi-fichiers, debug, migration
Copilot CLI	Élevé	Agent en ligne de commande	Shell, tests, CI, scripts

Copilot inline reste le point d'entrée : dès qu'on tape du code, des suggestions apparaissent en gris (`Tab` pour accepter). Voir le **Module 2**.

Copilot Chat couvre Ask, Edit, Plan et Agent. Voir le **Module 3**.

Instructions, Skills et mode Agent : voir les **Modules 4 et 5**.

Copilot CLI (`gh copilot`) reprend la logique agentique hors de l'éditeur — utile pour lancer les tests (`npm test`), typer (`tsc --noEmit`) ou formater le code.

1.3 Installation et configuration

Prérequis :

- Un compte GitHub avec un abonnement Copilot actif (Individual, Business ou Enterprise)
- Visual Studio Code installé
- Extension **TypeScript and JavaScript** de Microsoft (IntelliSense) + **ESLint** recommandé

Étapes d'installation :

1. Ouvrir VS Code
2. Aller dans Extensions (`Ctrl + Shift + X`)
3. Rechercher "GitHub Copilot" et installer l'extension

4. Installer également "GitHub Copilot Chat"
5. Se connecter à GitHub quand VS Code le demande
6. Vérifier l'icône Copilot dans la barre de statut (en bas)

Vérification du fonctionnement : Créer un fichier `test.ts` et commencer à taper :

```
// Fonction qui affiche Hello World
function greet(name: string): string {
```

Si Copilot fonctionne, une suggestion devrait apparaître en gris pour compléter la fonction.

1.4 Interface et statistiques d'utilisation

Pour consulter les statistiques d'utilisation de Copilot :

- Cliquer sur l'icône Copilot dans la barre de statut de VS Code
- Accéder au tableau de bord via GitHub : `Settings > Copilot > Usage`
- Les métriques disponibles : taux d'acceptation des suggestions, lignes de code générées, langages les plus utilisés

En entreprise (Copilot Business/Enterprise), les administrateurs ont accès à un dashboard détaillé montrant le pourcentage d'utilisation par équipe et par développeur.

Module 2 — Completions inline et contexte

Dans le module 1, nous avons installé Copilot. Les **completions inline** accélèrent l'écriture du boilerplate TypeScript (fonctions utilitaires, tests). Ce module approfondit raccourcis, contexte et commentaires-prompt.

Durée indicative : 3 h.

Objectifs

- Maîtriser les raccourcis clavier des suggestions inline.
- Expliquer la fenêtre de contexte et ses limites.
- Rédiger des commentaires-prompts (Quoi / Comment / Contraintes).
- Itérer : alternatives, acceptation partielle, reformulation.

Les **completions inline** sont le mode le plus utilisé au quotidien. Une fois les **Instructions** configurées (Module 4), elles produisent des suggestions alignées sur les standards du projet TypeScript.

> **Définition — Complétion inline**
>
> Suggestion affichée **en gris** pendant la frappe. Acceptez (`Tab`), rejetez (`Échap`) ou parcourez les alternatives.

2.1 Raccourcis clavier essentiels

ACTION	RACCOURCI (WINDOWS/LINUX)	RACCOURCI (MAC)
Accepter la suggestion	`Tab`	`Tab`
Rejeter la suggestion	`Échap`	`Échap`
Suggestion suivante	`Alt + ]`	`Option + ]`
Suggestion précédente	`Alt + [`	`Option + [`
Accepter le mot suivant	`Ctrl + →`	`Cmd + →`
Déclencher manuellement	`Alt + \`	`Option + \`
Ouvrir le panneau de suggestions	`Ctrl + Enter`	`Ctrl + Enter`

Le panneau de suggestions (`Ctrl + Enter`) ouvre une fenêtre avec jusqu'à 10 suggestions alternatives. Utile quand la première suggestion ne convient pas.

2.2 Déclencher des suggestions

Commencer à taper une signature de fonction

```
function calculateFactorial(n: number): number
```

Copilot va proposer le corps de la fonction en se basant sur le nom explicite et les types.

Écrire un commentaire descriptif

```
/** Tri à bulles sur un tableau de nombres, retourne une copie triée */
function bubbleSort(arr: number[]): number[]
```

Le commentaire guide Copilot sur l'algorithme attendu.

Créer une structure de données

```
interface Employee {
  name: string;
  age: number;
  salary: number;
}
```

Copilot pourra suggérer des fonctions cohérentes (createEmployee, formatEmployee, etc.).

Nommer une variable de manière explicite

```
const maxRetryCount = 3;
let errorMessage: string | null = null;
const inputStream = createReadStream("data.csv");
```

Des noms de variables clairs aident Copilot à comprendre l'intention du code.

2.3 Le contexte compte

Copilot ne se base pas uniquement sur la ligne en cours. Il analyse un **contexte élargi** :

Les fichiers ouverts dans l'éditeur : Si vous avez `utils.types.ts` ouvert avec des interfaces, Copilot les utilisera pour générer des implémentations cohérentes dans `utils.ts`.

Les imports influencent les suggestions :

```
import { Router } from "express";     // Copilot suggère des routes REST
import React, { useState } from "react"; // Copilot suggère des composants/hooks
import { z } from "zod";              // Copilot suggère des schémas de validation
```

Le code environnant guide la génération : Si les fonctions précédentes utilisent un style particulier (gestion d'erreurs, patterns async, immutabilité), Copilot va reproduire ce pattern.

```
// Si votre code existant fait ceci :
const result = await fetch(url);
if (!result.ok) {
  throw new ApiError(result.status, await result.text());
}

// Copilot reproduira ce pattern de gestion d'erreur HTTP
```

2.4 La fenêtre de contexte (context window)

La **fenêtre de contexte** (ou *context window*) est la quantité maximale de texte — code, commentaires, historique de chat, instructions du projet — que le modèle peut prendre en compte **en une seule requête**.

Définition concrète :

- Tout ce que Copilot « voit » avant de répondre occupe cette fenêtre : fichiers ouverts, sélection, messages du chat, `@workspace`, instructions Copilot, etc.
- Cette limite se mesure en **tokens** (morceaux de texte), pas en lignes de code.
- Au-delà de la limite, le contenu le plus ancien ou le moins prioritaire est **tronqué**.

Pourquoi c'est important :

CONSÉQUENCE	EXPLICATION
Perte de contexte	Un gros fichier + historique chat peuvent faire « oublier » le début.
Suggestions moins cohérentes	Si vos conventions ne tiennent plus dans la fenêtre, Copilot revient à des patterns génériques.
Réponses incomplètes en Agent	Sur un gros dépôt, l'agent doit cibler les bons fichiers.
Coût de qualité du prompt	Un contexte pertinent vaut mieux qu'un contexte volumineux.

Bonnes pratiques pour optimiser la fenêtre :

- Garder ouverts uniquement les fichiers **pertinents** (ex. `utils.types.ts` + `utils.ts`, pas tout le projet).
- Poser des questions **ciblées** dans le chat plutôt que de coller des milliers de lignes.
- Utiliser `#file:src/services/parser.ts` pour un fichier précis plutôt que `#workspace` quand la question est locale.
- Découper les grosses refontes en étapes (module par module).
- Centraliser les règles du projet dans les **Instructions** (voir Module 4).

2.5 L'art du commentaire-prompt

En TypeScript, les commentaires JSDoc et les signatures typées guident Copilot.

Commentaire vague → résultat imprécis :

```
// trier le tableau
```

Commentaire précis → résultat ciblé :

```
/**
 * Tri par insertion sur un tableau de nombres en ordre croissant.
 * Complexité O(n²) pire cas. Ne mute pas l'argument — retourne une copie.
 */
function insertionSort(arr: number[]): number[]
```

2.6 Principes de base

Être spécifique et précis :

```
// ❌ Vague
// lire un fichier

// ✅ Précis
/** Lit un fichier texte ligne par ligne. Retourne un tableau de lignes. Lance une erreur si le fichier est introuvable. */
async function readLines(filePath: string): Promise<string[]>
```

Donner du contexte :

```
// Cette fonction fait partie du service de cache Redis
// Récupère une clé avec TTL, retourne null si expirée ou absente
async function getCached<T>(key: string): Promise<T | null>
```

Décomposer les problèmes complexes :

```
// Étape 1 : Parser une ligne CSV en tokens
function parseCsvLine(line: string): string[];

// Étape 2 : Convertir les tokens en Employee
function tokenToEmployee(tokens: string[]): Employee;

// Étape 3 : Insérer dans le tableau avec immutabilité
function insertEmployee(employees: Employee[], emp: Employee): Employee[];
```

Itérer sur les suggestions : Si la première suggestion ne convient pas, utiliser `Alt + ]` pour voir les alternatives, ou reformuler le commentaire.

2.7 Structure d'un bon prompt

Un prompt efficace suit la structure **Quoi / Comment / Contraintes** :

```
/**
 * QUOI : Recherche un élément dans un tableau trié
 * COMMENT : Recherche dichotomique (binary search)
 * CONTRAINTES :
 *   - Tableau trié croissant
 *   - Retourne l'index ou -1
 */
function binarySearch(arr: readonly number[], target: number): number
```

Autre exemple :

```
/**
 * QUOI : Clone profond d'un arbre binaire typé
 * COMMENT : Parcours récursif avec spread immuable
 * CONTRAINTES :
 *   - Retourne null si la racine est null
 *   - Préserver le typage générique TreeNode<T>
 */
function deepCopyTree<T>(root: TreeNode<T> | null): TreeNode<T> | null
```

2.8 Itération et raffinement

Accepter partiellement une suggestion : Utiliser `Ctrl + →` (accepter mot par mot) quand le début de la suggestion est bon mais la suite diverge.

Modifier et relancer pour affiner :

```
// Premier essai — bubble sort basique
// Trier un tableau

// Deuxième essai — plus précis
/** Quicksort avec pivot médian, Lomuto, fallback insertion sort si length < 10 */
function quicksort(arr: number[]): number[]
```

Combiner plusieurs suggestions : Accepter une suggestion pour le squelette, puis supprimer certaines parties et redemander avec un commentaire plus spécifique.

Module 3 — Chat et modes d'interaction

Après les completions inline, ce module présente **Copilot Chat** pour diagnostiquer des bugs et planifier des refactorings TypeScript.

Durée indicative : 3 h.

Objectifs

- Utiliser les modes Ask, Edit, Plan et Agent.
- Sélectionner le contexte (@workspace, #file, sélection).
- Employer les commandes slash (/explain, /fix, /tests).
- Comprendre l'indexation sémantique du dépôt.

3.1 Les modes du Chat — Ask, Edit, Plan, Agent

Copilot Chat propose plusieurs **modes** selon le niveau d'autonomie souhaité. Ils partagent les **Instructions** du projet ; seuls **Agent** et partiellement **Ask** exploitent les **Skills** (voir Module 4).

MODE	AUTONOMIE	COMPORTEMENT	EXEMPLE EN TYPESCRIPT
Ask	Faible	Répond, explique, ne modifie pas les fichiers	« Explique ce hook useEffect et ses dépendances »
Edit	Moyenne	Modifie le code sélectionné ou le fichier actif	« Ajoute le typage strict sur ce paramètre any »
Plan	Moyenne	Produit un plan détaillé avant d'agir	« Plan pour migrer ce module CommonJS vers ESM »
Agent	Élevée	Planifie, édite, exécute, itère	« Corrige toutes les erreurs ESLint sur src/ »

Ask — comprendre du code sans modification :

- Sélectionner un bloc, poser une question : « Pourquoi cette Promise ne se résout jamais ? »
- Les Instructions s'appliquent (ex. réponse alignée sur vos conventions JSDoc / TSDoc)

Edit — changements localisés :

- Sélectionner une fonction, demander « /fix » ou une modification ciblée
- Plus rapide que l'Agent pour une modification ponctuelle

Plan — grosses tâches :

- Copilot produit un plan numéroté ; vous validez avant passage en Agent

Agent — cœur de l'approche agentique (voir Module 4) :

- Accès terminal, multi-fichiers, index sémantique
- Active automatiquement les **Skills** pertinents

3.2 Interface conversationnelle

Ouvrir le panneau Chat : `Ctrl + Shift + I` (ou `Cmd + Shift + I` sur Mac).

Exemples de questions utiles en TypeScript :

- "Explique-moi ce composant React et son cycle de vie"
- "Pourquoi TypeScript infère `never` ici ?"
- "Comment implémenter un middleware Express typé ?"
- "Génère les tests Vitest pour cette fonction"
- "Optimise ce rendu React pour éviter les re-renders inutiles"

Obtenir des explications détaillées : Sélectionner un bloc de code complexe puis demander dans le chat : "Explique ce code étape par étape, en particulier la gestion async/await et les types"

3.3 Commandes slash

COMMANDE	ACTION
`/explain`	Explique le code sélectionné
`/fix`	Propose une correction pour le code sélectionné
`/tests`	Génère des tests pour le code sélectionné
`/doc`	Génère la documentation (JSDoc / TSDoc)
`/new`	Crée un nouveau fichier/projet
`/clear`	Efface l'historique du chat

Exemple avec `/doc` :

```
function addNode(list: LinkedList, data: unknown): number;
```

```
/**
 * Ajoute un nœud en tête de la liste chaînée.
 * @param list - Liste cible
 * @param data - Données à stocker
 * @returns L'index du nœud créé, ou -1 en cas d'erreur
 */
function addNode(list: LinkedList, data: unknown): number;
```

3.4 Sélection de contexte

Surligner un bloc de code, puis ouvrir le chat → Copilot comprend que la question porte sur ce code précis.

3.5 Indexation sémantique du codebase

L'**indexation sémantique** permet à Copilot de **comprendre le sens** du code du projet, pas seulement de faire correspondre des mots-clés.

Principe :

- Le dépôt est analysé en embeddings : fonctions, types, commentaires, relations entre fichiers.
- Une question du type « Où est gérée la validation ? » ou `@workspace trouve les handlers dupliqués` s'appuie sur cette index.
- Les résultats pertinents sont injectés dans la fenêtre de contexte.

APPROCHE	LIMITE
Fichiers ouverts + ligne courante	Ne couvre que ce que vous avez sous les yeux
Recherche par nom de symbole	Rate les implémentations sous un autre nom
Index sémantique	Retrouve du code par **intention** (« validation Zod », « middleware auth », « gestion d'erreur API »)

Bonnes pratiques :

- Laisser l'indexation se terminer après un clone ou un gros pull.
- Formuler des requêtes avec des **concepts** (« hook React », « parsing CSV », « repository pattern »).
- Combiner index sémantique + `#file:src/services/parser.ts`.

3.6 Mode Agent en pratique

Le **mode Agent** est le point d'application des **Skills** et **Instructions** configurés au Module 4.

Copilot peut :

- Exécuter des commandes terminal (`npm test`, `tsc --noEmit`, `eslint`, `vitest`)
- Modifier plusieurs fichiers en séquence
- Itérer jusqu'à résoudre un problème ou signaler un blocage

```
Mode Agent : « Corrige toutes les erreurs TypeScript et ESLint sur src/ »
→ Copilot active le skill lint-and-typecheck (si présent)
→ Applique les Instructions (strict, pas de any)
→ Lance tsc et eslint, corrige les fichiers, relance les tests
```

Bonnes pratiques Agent :

- Formuler un **objectif mesurable** (« 0 erreur tsc et eslint sur src/ »)
- Laisser l'indexation sémantique se terminer sur les gros dépôts
- Vérifier manuellement le diff avant commit

3.7 Mode Cloud (aperçu)

Le mode Cloud permet d'exécuter des tâches Copilot sur l'infrastructure GitHub :

- Tâches longues en arrière-plan
- Pas besoin de garder VS Code ouvert
- Résultats via notification ou PR
- Utile pour des refactorings massifs ou des migrations

Module 4 — Agent, Skills et Instructions

Dans le module 3, nous avons exploré le Chat. Ce module détaille la **personnalisation agentique** : Instructions, Skills et mode Agent.

Durée indicative : 4 h.

Objectifs

- Rédiger `.github/copilot-instructions.md` adapté au projet TypeScript.
- Créer un Skill métier (lint, tests, typecheck).
- Lire le schéma Instructions → Skills → Agent.
- Tester la personnalisation en mode Agent.

Définition — Instructions

Règles **permanentes** du dépôt, injectées à chaque interaction (inline, chat, agent). Fichier principal : `.github/copilot-instructions.md`.

Définition — Instructions

Règles **permanentes** du dépôt, injectées à chaque interaction (inline, chat, agent). Fichier : `.github/copilot-instructions.md`.

4.1 Vue d'ensemble

CONCEPT	RÔLE	QUAND C'EST ACTIF	FICHIER TYPIQUE
Instructions	Règles permanentes du projet	**Toujours** (inline, chat, agent)	`.github/copilot-instructions.md`
Skill	Workflow spécialisé, chargé à la demande	Quand la tâche correspond à la description	`.github/skills/<nom>/SKILL.md`
Agent	Mode autonome qui planifie et exécute	Sur demande explicite (mode Agent)	Panneau Chat ou Copilot CLI

Les **Instructions** définissent *comment coder dans ce dépôt*. Les **Skills** enseignent *comment accomplir une tâche répétitive*. L'**Agent** *orchestre* le tout.

	INSTRUCTIONS	SKILL
Contenu	Règles courtes, standards du projet	Workflow détaillé, scripts, références
Activation	Toujours	Seulement si la tâche est pertinente

4.2 Instructions globales — `.github/copilot-instructions.md`

Fichier à la racine du dépôt (dossier `.github/`). Copilot l'injecte dans **toutes** les interactions.

FICHIER	PORTÉE
`.github/copilot-instructions.md`	Global — tout le dépôt
`.github/instructions/*.md`	Par chemin (`applyTo` en en-tête YAML)
Instructions personnelles	GitHub → Settings → Copilot (tous vos projets)

Exemple :

```
# Instructions — projet TypeScript

- TypeScript `strict: true` ; éviter `any` sur l'API publique
- camelCase variables/fonctions ; PascalCase types et composants React
- `async/await` plutôt que chaînes `.then()`
- JSDoc sur l'API publique (`@param`, `@returns`)
- Erreurs typées ou `Result<T, E>` — pas de `throw` nu sans contexte
- Avant toute suggestion : vérifier mentalement `tsc --noEmit`
```

Bonnes pratiques :

- Rester **court** (≤ 200 lignes) — le détail va dans un Skill ou une instruction path-specific (module 5).
- Documenter *pourquoi* une règle existe, pas seulement *quoi*.
- Consignes pédagogiques : ne pas compléter les zones à implémenter des exercices.

> *Les Instructions restent **générales**. Pour un workflow détaillé, préférer un **Skill**.*

4.3 Skills — `.github/skills/<nom>/SKILL.md`

Définition — Skill

Dossier avec `SKILL.md` (frontmatter `name`, `description`). Copilot charge le skill **quand la tâche correspond** à la description YAML.

Structure :

```
.github/skills/
└── lint-and-check/
    ├── SKILL.md
    ├── scripts/
    └── references/
```

Exemple `SKILL.md` **:**

```
---
name: lint-and-check
description: Vérifie le typage strict et ESLint. Utiliser quand l'utilisateur
mentionne tsc, ESLint, type error, lint ou strict mode.
---

## Workflow

1. Identifier les scripts (`npm run lint`, `npm run typecheck`)
2. Exécuter `tsc --noEmit` puis `eslint src/`
3. Corriger types/imports fichier par fichier
4. Relancer `vitest run` ou `npm test`
```

ACTION	DOCUMENTATION
Créer un skill	About agent skills
Instructions dépôt	Repository custom instructions

4.4 Mode Agent en pratique

Le **mode Agent** (Module 3) est le point d'application des Instructions et Skills configurés ci-dessus.

Capacités :

- Exécuter des commandes terminal (`npm test, tsc --noEmit, eslint`)
- Modifier plusieurs fichiers en séquence
- Itérer jusqu'à un objectif mesurable ou signaler un blocage

Exemple :

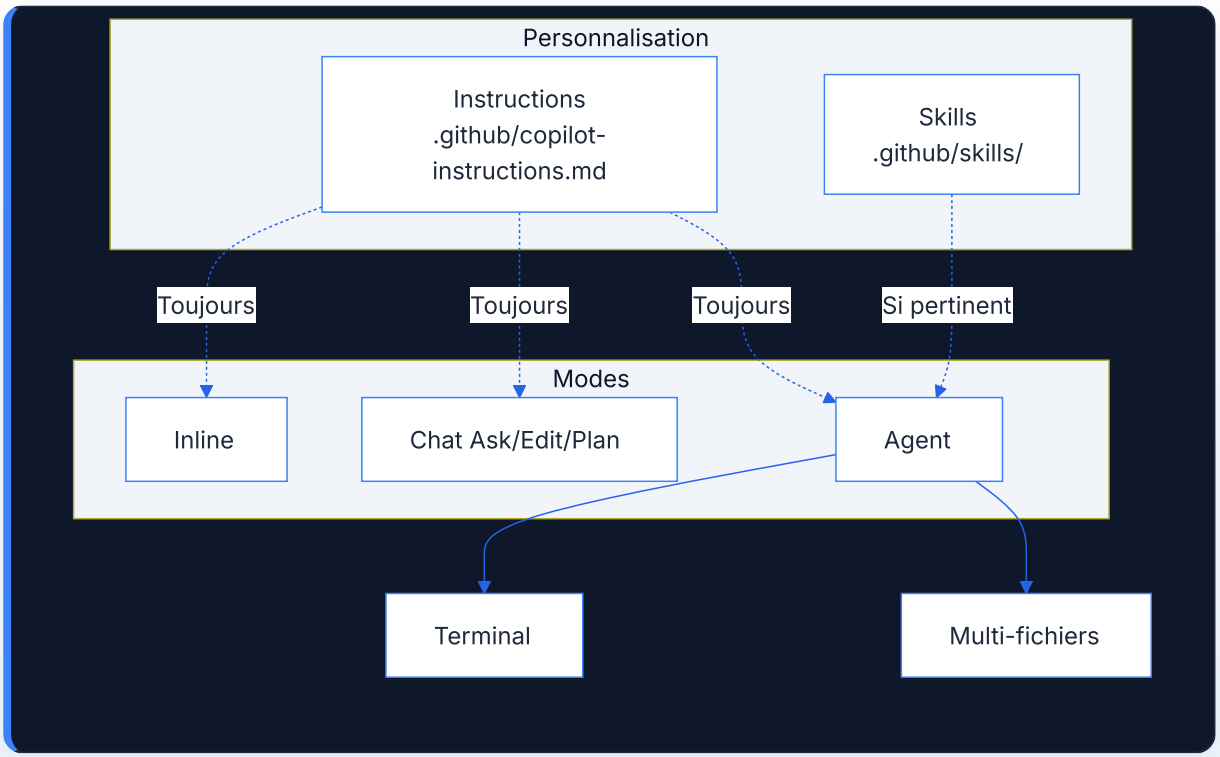
```
Mode Agent : « Corrige toutes les erreurs TypeScript strict et ESLint sur src/ »
→ Lance tsc et eslint, corrige les fichiers, relance les tests
```

Bonnes pratiques :

- Objectif **mesurable** (« 0 warning gcc », « tests verts »)
- Vérifier le diff avant commit
- Laisser l'indexation sémantique se terminer sur les gros dépôts

4.5 Schéma explicatif — comment tout s'articule

GRAPHIQUE 2



Lecture : Instructions = socle permanent ; Skills = workflows à la demande ; Agent = exécution autonome.

4.6 Mise en place minimale

1. **Instructions globales** — `.github/copilot-instructions.md`
2. **Instructions tests** — `.github/instructions/tests.md` avec `applyTo: "**/*.spec.ts,**/*.test.ts"`
3. **Un skill métier** — `.github/skills/lint-and-check/SKILL.md`

4. **Tester en Agent** — `@workspace Corrige les erreurs TypeScript strict et ESLint sur src/`

4.7 Prompts réutilisables (complément)

Fichiers `.github/prompts/*.prompt.md` — modèles de demandes récurrentes (complément aux Skills).

```
<!— .github/prompts/new-module.prompt.md —>
Crée un nouveau module avec :
- Fichier `index.ts` exportant l'API publique
- Types/interfaces dans `types.ts`
- Tests Vitest dans `__tests__/`
```

Module 5 — Path-specific, commit et review

Dans le module 4, nous avons posé les Instructions globales. Ce module montre comment affiner Copilot **par zone du dépôt** : API, tests et code TypeScript n'obéissent pas aux mêmes règles.

Durée indicative : 3 h.

Objectifs

- Configurer des instructions path-specific (applyTo).
- Créer des Skills avancés avec scripts.
- Générer des messages de commit avec Copilot.
- Lancer une revue de code assistée.

Les concepts **Agent**, **Skill** et **Instruction** sont détaillés au **Module 4**.

5.1 Instructions spécifiques par chemin (path-specific)

Les instructions **path-specific** ne s'activent que lorsque Copilot travaille sur des fichiers correspondant à un **glob**.

Pourquoi les utiliser :

- Le front React (`src/components/`) n'a pas les mêmes règles qu'une API Node (`src/api/`)
- Les tests (`*.spec.ts`) peuvent autoriser des mocks interdits en production
- Les fichiers `.tsx` vs `.ts` peuvent demander des conventions distinctes

Organisation recommandée :

```
---
applyTo: "src/**/*.ts,src/**/*.tsx"
---

# Règles pour le code source TypeScript

- strict mode, pas de `any`
- Préférer les types inférés sauf API publique
- Composants React : PascalCase, props typées avec interface dédiée
```

```
---
applyTo: "**/*.spec.ts,**/*.test.ts"
---

# Règles pour les tests

- Vitest ou Jest selon le projet existant
- Utiliser describe/it et les helpers du projet
- Mocker uniquement les dépendances externes, pas la logique testée
```

```
---
applyTo: "exercices/**/*.ts"
---

# Contexte pédagogique — exercices étudiants

- Laisser les blocs TODO intacts
- Suggérer des indices en commentaire plutôt que des solutions complètes
- Respecter les signatures imposées par l'énoncé
```

Ordre de priorité :

1. Instructions globales
2. Instructions path-specific (glob correspondant)
3. Contexte immédiat (fichier, sélection, commentaire-prompt)

Bonnes pratiques :

- Préférer des globs **étroits** (`src/api/*.ts`) à `**/*`
- Documenter *pourquoi* chaque règle existe
- Vérifier qu'une consigne globale n'annule pas une consigne locale

Créer des Skills avancés :

- Scripts dans `scripts/` (wrapper `npm run lint` avec options du projet)
- Doc lourde dans `references/` pour préserver la fenêtre de contexte
- Affiner la `description` YAML — **déclencheur** de sélection

5.2 Prompts réutilisables (`.github/prompts/`)

```
<!-- .github/prompts/new-module.prompt.md -->

Crée un nouveau module TypeScript avec :

- Un fichier `index.ts` exportant l'API publique
- Types/interfaces dans `types.ts`
- Tests Vitest dans `__tests__/`
- JSDoc sur chaque export public
```

5.3 Commit automatique

- Cliquer sur l'icône Copilot dans Source Control
- Copilot analyse le diff et propose un message (Conventional Commits si configuré)

Exemple :

```
feat(parser): add CSV parsing with quoted field support

- Handle escaped quotes within fields
- Support multiline values enclosed in quotes
- Add error reporting with line numbers
```

5.4 Code review sur les commits en cours

- Source Control → "Review Changes" avec Copilot
- Particulièrement utile en TypeScript pour détecter :
- Usages de `any` ou assertions de type dangereuses
- Promises non gérées (floating promises)
- Fuites mémoire côté Node (listeners, timers)
- XSS via `dangerouslySetInnerHTML` ou injection dans les templates

5.5 Fine tuning et personnalisation

- Les **Instructions** (Module 4) influencent toutes les suggestions
- Les **Skills** standardisent les workflows répétitifs
- Copilot apprend des patterns du code existant dans le dépôt

---

Module 6 — Bonnes pratiques et productivité

Dans le module 5, nous avons affiné la gouvernance. Ce module clôt la formation : **quand** faire confiance à Copilot et **comment** valider le code TypeScript généré.

Durée indicative : 2 h.

Objectifs

- Appliquer une checklist de validation sur le code généré.
- Identifier les cas d'usage adaptés (et inadaptés) à Copilot.
- Adopter un workflow de productivité durable.

- Formaliser les bonnes pratiques d'équipe.

6.1 Validation du code généré

Le code TypeScript généré par Copilot nécessite une vigilance particulière :

Toujours vérifier :

- Le typage (pas de `any` dissimulé, generics corrects)
- La gestion async (await, erreurs, race conditions)
- Les null/undefined (`strictNullChecks`)
- Les dépendances React (tableau de deps useEffect/useMemo)
- La sécurité (sanitisation entrées, secrets en dur)

Outils de validation :

```
# Vérification de types
npx tsc --noEmit

# Lint
npx eslint src/

# Tests
npm test
# ou
npx vitest run
```

6.2 Quand utiliser Copilot

Code répétitif ou boilerplate :

```
// Copilot excelle pour générer des fonctions CRUD similaires
function createEmployee(data: CreateEmployeeDto): Employee;
function updateEmployee(id: string, data: UpdateEmployeeDto): Employee;
function deleteEmployee(id: string): void;
function getEmployeeById(id: string): Promise<Employee | null>;
```

Algorithmes classiques, tests, exploration d'APIs : Copilot connaît les implémentations standards et le boilerplate (React, Express, Zod, Prisma).

Tests unitaires :

```
// Demander : "Génère les tests Vitest pour binarySearch"
describe("binarySearch", () => {
  it("finds existing element", () => {
    expect(binarySearch([1, 3, 5, 7, 9], 7)).toBe(3);
  });
  it("returns -1 when not found", () => {
    expect(binarySearch([1, 3, 5, 7, 9], 4)).toBe(-1);
  });
});
```

6.3 Quand être prudent

Code critique pour la sécurité : toujours relire et tester en profondeur.

Logique métier complexe : Copilot peut produire du code syntaxiquement correct mais sémantiquement faux.

Contraintes spécifiques : SSR, edge runtime, bundles optimisés — Copilot ne connaît pas toujours les contraintes de déploiement.

Performance critique : Copilot génère du code fonctionnel mais rarement optimal. Pour des boucles critiques ou du rendu React lourd, l'expertise humaine reste indispensable.

6.4 Productivité optimale

- Lire et comprendre chaque suggestion avant de l'accepter
- Copilot accélère l'écriture, il ne dispense pas de la réflexion
- Observer les suggestions pour découvrir des idiomes (découvrir `structuredClone`, `Array.at()`, les utility types (`Partial`, `Pick`) via les suggestions)

Adapter son workflow progressivement :

1. **Completions inline** — boilerplate (Module 2)
2. **Chat Ask/Edit** — debugging et documentation (Module 3)
3. **Instructions** — `.github/copilot-instructions.md` (Module 4)
4. **Skills** — workflow récurrent (Module 5)

5. **Mode Agent** — tâches multi-fichiers avec relecture du diff